

SIMATS ENGINEERING

Assessment -CO5-AT1

CO5: Design AI-based systems using PySpark, RDD operations, and intelligent agent architectures,

using scalable systems and integrate components in distributed AI applications. (BTL 5)

Subject	Artificial Intelligence
Course Code	CSA 1712
Name	Vignesh Kumar K
Register Number	192512375

Declaration

I, Vignesh Kumar K (Reg No: 192512375), certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

Design Task — Hybrid AI Recommendation Engine

Five design-task options were provided in the assessment, sharing one common evaluation rubric. This response addresses the **Hybrid AI Recommendation Engine** scenario in full depth: "Design a hybrid AI-based recommendation system that integrates content-based and collaborative filtering techniques using PySpark RDDs. Explain how distributed data processing can be used to handle large-scale user-item interactions, and describe the role of intelligent agents in adapting recommendations based on user behavior and feedback in real time."

1. Problem Understanding & Requirement Analysis

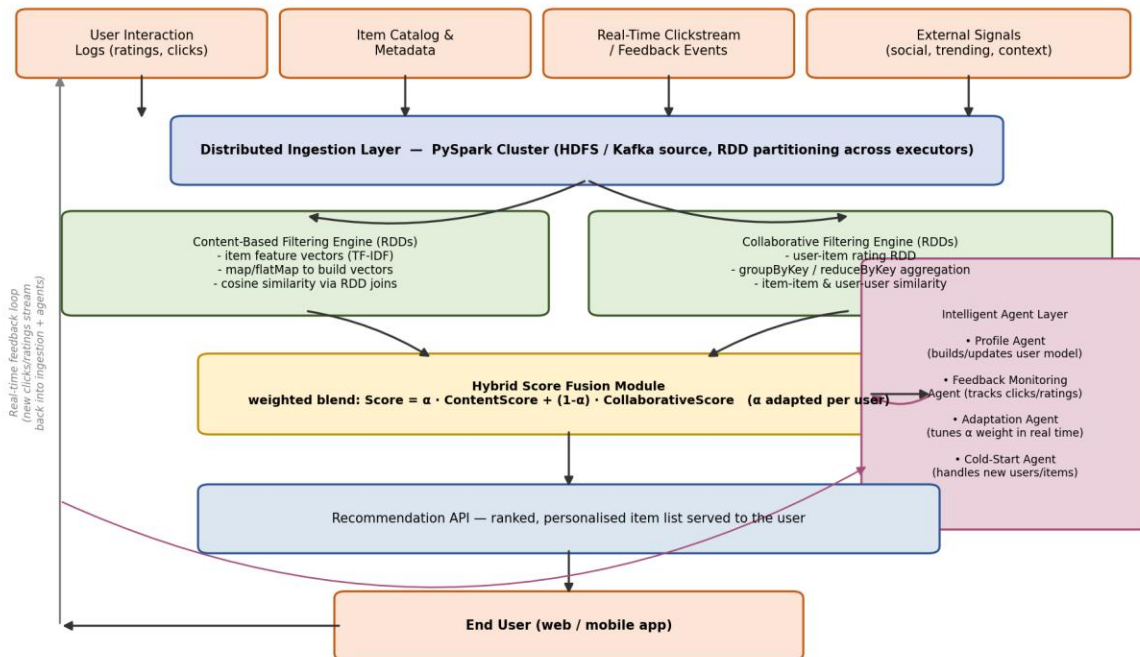
Objective: build a recommendation engine that suggests relevant items (products, videos, courses, etc.) to users at scale, combining two complementary recommendation paradigms so their individual weaknesses cancel out.

Requirement	Analysis
Content-Based Filtering	Recommends items similar to what a user has liked before, using item attributes (genre, category, description/TF-IDF features). Works well for new items (no interaction history needed) but tends to over-specialise and cannot capture 'users like you also liked...' patterns.
Collaborative Filtering	Recommends items liked by users with similar behaviour/rating patterns. Captures cross-item, cross-user patterns content-based filtering misses, but suffers badly from the 'cold-start' problem for brand-new users or items with no interaction history yet.
Scale requirement	The platform must process potentially millions of user-item interactions and a large, frequently-updated item catalog — a single-machine solution cannot process or store this volume within acceptable latency, motivating a distributed (PySpark/RDD) design.
Real-time adaptation requirement	User preferences and context drift constantly (mood, trends, session intent) — a static, periodically-retrained model alone is insufficient; the system must incorporate real-time feedback signals (clicks, dwell time, explicit ratings) to adapt recommendations within the same session.

Constraints identified: large and growing data volume (necessitates horizontal scalability), the cold-start problem (necessitates a hybrid approach rather than collaborative filtering alone), latency requirements for real-time re-ranking (necessitates a streaming/agent layer, not just a nightly batch job), and fault tolerance (a production recommender cannot go down because one executor node fails).

2. System Architecture Design

Hybrid AI Recommendation Engine — System Architecture



The architecture is organised into six layers:

- **1. Data Sources:** User interaction logs, item catalog/metadata, real-time clickstream/feedback events, and optional external signals (social/trending context) — heterogeneous sources feeding the pipeline.
- **2. Distributed Ingestion Layer:** A PySpark cluster reads raw data from HDFS/S3 (historical batch data) and Kafka (real-time events), immediately partitioning it into RDDs across the cluster's executors so no single node needs to hold the full dataset in memory.
- **3. Dual Processing Engines:** Two parallel RDD pipelines run independently: the Content-Based Filtering Engine (item feature vectors + cosine similarity) and the Collaborative Filtering Engine (user-item matrix + similarity/ALS-style factorization) — designed to run concurrently on separate partitions of the cluster.
- **4. Hybrid Score Fusion Module:** Combines both engines' outputs into one ranked score per candidate item using a weighted blend, where the blending weight α is not fixed but tuned per-user by the Agent Layer (see Section 4).
- **5. Intelligent Agent Layer:** Four cooperating agents (Profile, Feedback Monitoring, Adaptation, Cold-Start) sit alongside the fusion module, continuously consuming the real-time feedback stream and adjusting how recommendations are generated — detailed in Section 4.
- **6. Recommendation API & Feedback Loop:** Serves the final ranked list to the user interface; every new click/rating the user generates flows back into the ingestion layer and directly into the Agent Layer, closing the real-time adaptation loop.

3. Use of PySpark & RDD Operations

Distributed processing is essential here because user-item interaction data (potentially billions of rows for a large platform) cannot fit in — or be processed efficiently by — a single machine. PySpark RDDs partition this data across a cluster and process each partition in parallel, with Spark's DAG scheduler automatically handling task distribution and re-execution of failed tasks.

3.1 Ingestion and Cleaning

```
raw_rdd = sc.textFile('hdfs:///data/interactions/*.csv')
parsed_rdd = raw_rdd.map(lambda line: parse_interaction(line))
clean_rdd = parsed_rdd.filter(lambda r: r is not None and r.rating is not None).cache()
```

`.cache()` persists the cleaned RDD in memory across the cluster, since it is reused by both downstream engines — avoiding redundant re-computation of the same transformations.

3.2 Content-Based Filtering (RDD transformations)

```
# Build a TF-IDF-like feature vector per item from its metadata/description
item_features_rdd = items_rdd.map(lambda item: (item.id, build_feature_vector(item)))

# Pairwise item-item similarity (cosine) via a cartesian join, then map to similarity scores
content_sim_rdd = (item_features_rdd
                  .cartesian(item_features_rdd)
                  .filter(lambda pair: pair[0][0] != pair[1][0])
                  .map(lambda pair: ((pair[0][0], pair[1][0]),
                                     cosine_similarity(pair[0][1], pair[1][1]))))
```

3.3 Collaborative Filtering (RDD transformations)

```
# (user, (item, rating)) pairs, grouped per user to compute similarity between users' rating
vectors
user_item_rdd = clean_rdd.map(lambda r: (r.user_id, (r.item_id, r.rating)))
grouped_rdd = user_item_rdd.groupByKey().mapValues(list)

# reduceByKey aggregation example: average rating per item (used for popularity-based
fallback)
item_avg_rdd = (clean_rdd.map(lambda r: (r.item_id, (r.rating, 1)))
               .reduceByKey(lambda a, b: (a[0]+b[0], a[1]+b[1]))
               .mapValues(lambda x: x[0] / x[1]))

# user-user or item-item similarity can equally be computed via Spark MLlib's ALS
# on top of the same user_item_rdd for large-scale matrix factorization
```

3.4 Hybrid Fusion (join + mapValues)

```
hybrid_rdd = (content_sim_rdd.join(collaborative_sim_rdd)
              .mapValues(lambda scores: alpha*scores[0] + (1-alpha)*scores[1]))

ranked_rdd = hybrid_rdd.sortBy(lambda x: -x[1])
top_n = ranked_rdd.take(10)    # action: triggers the DAG execution, returns to the driver
```

Key RDD operations used throughout: **map / flatMap** (per-record transforms), **filter** (data cleaning), **groupByKey / reduceByKey** (per-user/per-item aggregation), **join** (combining the two engines' outputs), **cache/persist** (reusing hot RDDs), and **sortBy / take** (final ranking actions that trigger lazy DAG execution).

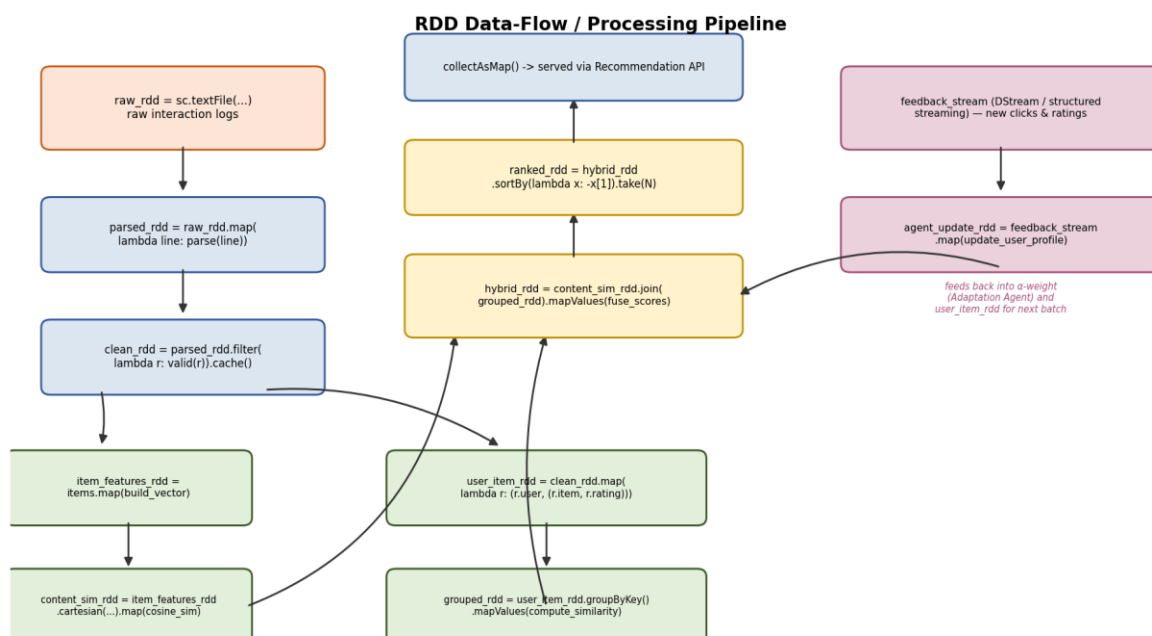
4. Intelligent Agent Integration

Four cooperating agents adapt the static hybrid model into a system that reacts to user behaviour in real time:

Agent	Type	Role & Logic
Profile Agent	Model-based	Continuously updates each user's internal profile (preference vector, recent categories browsed) as new interactions arrive from the feedback stream, giving the fusion module up-to-date input rather than a stale batch snapshot.
Feedback Monitoring Agent	Simple reflex	Watches the real-time click/rating stream and immediately flags strong signals (e.g., explicit 'not interested', or a rapid sequence of clicks in one category) that should influence the very next recommendation, without waiting for the next batch cycle.
Adaptation Agent	Utility-based	Dynamically tunes the fusion weight α per user: it increases the collaborative-filtering weight for users with rich interaction history (where collaborative signals are reliable) and increases the content-based weight for sparse-history users — maximizing a utility function balancing recommendation relevance against exploration.
Cold-Start Agent	Goal-based	Specifically targets the goal of giving new users/items reasonable recommendations despite having no collaborative signal yet, by initially relying almost entirely on content-based similarity and popularity fallbacks, and handing off to the Adaptation Agent once enough interaction history accumulates.

These agents interact with the PySpark pipeline through the streaming feedback branch: incoming events are processed via Spark Structured Streaming (or DStreams) into micro-batches, each of which updates the relevant agent's internal state and, where warranted, triggers an incremental re-ranking rather than waiting for the next full batch recomputation — giving the system real-time responsiveness without discarding the efficiency of batch processing for the bulk of the similarity computation.

5. Data Flow & Processing Pipeline



The pipeline is intentionally lazy: none of the *map*, *filter*, *groupByKey*, or *join* transformations actually execute when called — Spark only builds a DAG (directed acyclic graph) of the computation. Execution is triggered only by an **action** (e.g., *take(N)* or *collectAsMap()*) at the end of the pipeline, at which point Spark's scheduler optimises and distributes the entire chain of transformations across the cluster in one pass — avoiding unnecessary intermediate materialisation of data.

6. Scalability & Distributed Computing Design

Design Concern	How the Architecture Addresses It
Parallelism	RDDs are automatically split into partitions distributed across executors; the number of partitions is tuned (e.g., <i>repartition(200)</i>) to match cluster core count, so similarity computations for millions of item pairs run concurrently rather than sequentially.
Data partitioning strategy	<i>user_item_rdd</i> is partitioned by <i>user_id</i> hash (<i>partitionBy(HashPartitioner(N))</i>), so that all of a given user's interactions land on the same partition — this makes the <i>groupByKey</i> aggregation for that user a local operation, avoiding expensive network shuffles.
Fault tolerance	RDD lineage means Spark can automatically recompute any lost partition from its parent transformations if an executor node fails, without needing manual checkpoint/restart logic; periodic <i>checkpoint()</i> calls are also used on long transformation chains (e.g., the iterative similarity computation) to bound recovery cost.
Caching / memory management	Frequently reused RDDs (<i>clean_rdd</i> , <i>item_features_rdd</i>) are cached in memory (or spilled to disk under memory pressure) to avoid recomputing the same transformation chain multiple times across the content-based and collaborative branches.
Broadcast variables	Small, frequently-referenced lookup data (e.g., the item metadata table) is distributed via broadcast variables rather than being shuffled/joined repeatedly, since every executor needs read access to the same small table.
Handling data skew	A small number of extremely popular items can create a 'hot partition' during <i>groupByKey</i> ; salting the key (e.g., appending a random suffix and aggregating in two stages) prevents any single executor from becoming a bottleneck.

7. Innovation & Creativity

- Adaptive per-user hybrid weighting (α) rather than a single global content/collaborative blend ratio — the Adaptation Agent effectively personalises not just the recommendations but the recommendation strategy itself per user.
- A dedicated Cold-Start Agent with an explicit hand-off criterion to the Adaptation Agent once a user accumulates enough history, rather than a single fixed heuristic threshold applied uniformly.
- Treating real-time feedback as a first-class streaming branch of the same RDD-based pipeline (via Structured Streaming micro-batches) rather than a completely separate real-time system bolted on afterward — keeping the codebase and data model unified between batch and real-time paths.
- Salted-key aggregation to pre-emptively address popularity-driven data skew, an issue many simpler recommender designs overlook until it causes production incidents.

8. Summary

This design satisfies the brief's requirements end-to-end: a hybrid content-based + collaborative recommendation engine built on PySpark RDD transformations (map, filter, groupByKey, reduceByKey, join, cache), explicitly engineered for horizontal scalability and fault tolerance across a distributed cluster, and augmented with a four-agent intelligent layer that adapts recommendations to individual user behaviour and feedback in real time rather than relying solely on periodic batch retraining.